МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

ОТЧЁТ

по лабораторной работе № 2

«Базовые SQL-запросы и фильтрация данных»

по дисциплине «Математические основы баз данных»

Тема сквозного кейса: «Цифровая платформа торговли кибернетическими имплантами и био-модификациями “КИиБ”»

Выполнил студент группы БВТ2505

Гущин Михаил Борисович

Проверил: Быватов Семен Юрьевич

Москва 2026

1. НАПОЛНЕНИЕ БАЗЫ ДАННЫХ

1.1 Подход к наполнению

База, спроектированная в лабораторной работе №1 (10 таблиц, схема KIIB), наполнена тестовыми данными. Часть строк внесена вручную — это «флагманские» позиции с узнаваемыми названиями (Kiroshi Optics Mk.3, Sandevistan Mk.4, Mantis Blades и т.п.) и восемь продавцов-корпораций. Остальной объём, прежде всего товары и заказы, сгенерирован функцией generate_series в сочетании с random(), как и рекомендует методическое пособие.

При генерации случайных значений пришлось учитывать особенность планировщика PostgreSQL: если подзапрос, отбирающий случайную строку, не ссылается ни на один столбец внешнего запроса, планировщик считает его результат независимым от внешних строк и вычисляет один раз, подставляя одно и то же значение во все строки. Поэтому случайный выбор продавца, категории и товара сделан не отдельным подзапросом, а прямым выражением в списке SELECT — индексацией по массиву (array_agg(id)), так функция random() пересчитывается для каждой строки результата отдельно.

1.2 Объёмы данных

Таблица	Строк	Способ наполнения
`users`	200	16 именных + generate_series
`sellers`	8	вручную (8 корпораций/клиник)
`categories`	11	вручную (4 корневые + 7 дочерних)
`products`	1000	10 флагманских + generate_series
`stock`	1000	по одной строке на товар
`orders`	1200	generate_series
`order_items`	≈ 3600	1–3 позиции на заказ, случайный товар
`reviews`	≈ 900	случайные пары (пользователь, товар)
`payments`	≈ 900	по заказам со статусом ≠ pending
`addresses`	≈ 130	для части пользователей и продавцов

	t text	count bigint
1	users	200
2	sellers	8
3	categories	11
4	products	1000
5	stock	1000
6	orders	1200
7	order_ite...	3593
8	reviews	897
9	payments	907
10	addresses	126

```
-- ============================================================
-- ПРОВЕРКА ОБЪЁМОВ
-- ============================================================
SELECT 'users' t, count(*) FROM users
UNION ALL SELECT 'sellers', count(*) FROM sellers
UNION ALL SELECT 'categories', count(*) FROM categories
UNION ALL SELECT 'products', count(*) FROM products
UNION ALL SELECT 'stock', count(*) FROM stock
UNION ALL SELECT 'orders', count(*) FROM orders
UNION ALL SELECT 'order_items', count(*) FROM order_items
UNION ALL SELECT 'reviews', count(*) FROM reviews
UNION ALL SELECT 'payments', count(*) FROM payments
UNION ALL SELECT 'addresses', count(*) FROM addresses;
```

2. ЗАПРОСЫ НА ВЫБОРКУ И ФИЛЬТРАЦИЮ

2.1 DISTINCT и псевдонимы столбцов

Запрос выводит уникальные комбинации названия и цены товара, вычисляет цену со скидкой 10% и даёт всем трём столбцам псевдонимы. Результат отсортирован по убыванию цены — видно, что дороже всего обходятся Sandevistan Mk.4 и Mantis Blades.

```
SELECT DISTINCT
    title       AS name,
    price,
    round(price * 0.9, 2) AS discounted_price
FROM products
ORDER BY price DESC
LIMIT 20;
```

Query | Query History

```
1  -- 2.1 DISTINCT + псевдонимы столбцов
2  SELECT DISTINCT
3      title       AS name,
4      price,
5      round(price * 0.9, 2) AS discounted_price
6  FROM products
7  ORDER BY price DESC
8  LIMIT 20;
9
```

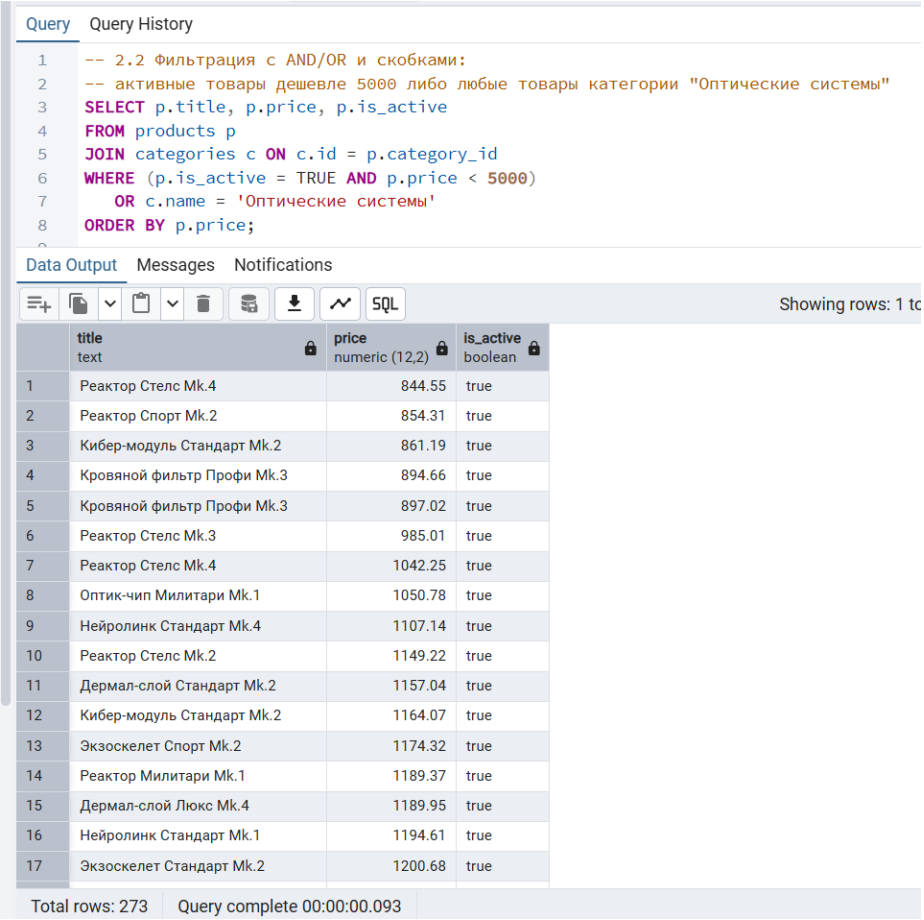
Data Output | Messages | Notifications

	name text	price numeric (12,2)	discounted_price numeric
1	Sandevistan Mk.4	45000.00	40500.00
2	Mantis Blades	38000.00	34200.00
3	Cyberdeck Tetratronic Rippers...	32000.00	28800.00
4	Gorilla Arms	27500.00	24750.00
5	Дермал-слой Стандарт Mk.4	24781.53	22303.38
6	Кибер-модуль Спорт Mk.3	24767.01	22290.31
7	Кибер-модуль Профи Mk.3	24732.11	22258.90
8	Оптик-чип Спорт Mk.4	24695.56	22226.00
9	Экзоскелет Спорт Mk.4	24693.63	22224.27
10	Кровяной фильтр Стелс Mk.4	24676.50	22208.85
11	Кровяной фильтр Спорт Mk.5	24616.33	22154.70
12	Нейролинк Спорт Mk.2	24594.83	22135.35
13	Кибер-модуль Милитари Mk.4	24586.39	22127.75
14	Кибер-модуль Люкс Mk.2	24552.07	22096.86
15	Реактор Милитари Mk.2	24545.77	22091.19
16	Нейролинк Милитари Mk.1	24512.33	22061.10
17	Дермал-слой Милитари Mk.1	24495.13	22045.62
18	Кровяной фильтр Стелс Mk.3	24493.98	22044.58
19	Нейролинк Стандарт Mk.4	24491.23	22042.11
20	Реактор Спорт Mk.2	24445.18	22000.66

2.2 Фильтрация с AND/OR и скобками

Условие объединяет два разных критерия через OR: либо активный и дешёвый товар (цена < 5000), либо любой товар категории «Оптические системы» — независимо от цены и активности. Скобки нужны, чтобы AND внутри первого условия не «расползся» на всё выражение. Запрос вернул 273 строки.

```
SELECT p.title, p.price, p.is_active
FROM products p
JOIN categories c ON c.id = p.category_id
WHERE (p.is_active = TRUE AND p.price < 5000)
   OR c.name = 'Оптические системы'
ORDER BY p.price;
```



Query | Query History

```
1  -- 2.2 Фильтрация с AND/OR и скобками:
2  -- активные товары дешевле 5000 либо любые товары категории "Оптические системы"
3  SELECT p.title, p.price, p.is_active
4  FROM products p
5  JOIN categories c ON c.id = p.category_id
6  WHERE (p.is_active = TRUE AND p.price < 5000)
7     OR c.name = 'Оптические системы'
8  ORDER BY p.price;
```

Data Output | Messages | Notifications

Showing rows: 1 to

	title text	price numeric (12,2)	is_active boolean
1	Реактор Стелс Mk.4	844.55	true
2	Реактор Спорт Mk.2	854.31	true
3	Кибер-модуль Стандарт Mk.2	861.19	true
4	Кровяной фильтр Профи Mk.3	894.66	true
5	Кровяной фильтр Профи Mk.3	897.02	true
6	Реактор Стелс Mk.3	985.01	true
7	Реактор Стелс Mk.4	1042.25	true
8	Оптик-чип Милитари Mk.1	1050.78	true
9	Нейролинк Стандарт Mk.4	1107.14	true
10	Реактор Стелс Mk.2	1149.22	true
11	Дермал-слой Стандарт Mk.2	1157.04	true
12	Кибер-модуль Стандарт Mk.2	1164.07	true
13	Экзоскелет Спорт Mk.2	1174.32	true
14	Реактор Милитари Mk.1	1189.37	true
15	Дермал-слой Люкс Mk.4	1189.95	true
16	Нейролинк Стандарт Mk.1	1194.61	true
17	Экзоскелет Стандарт Mk.2	1200.68	true

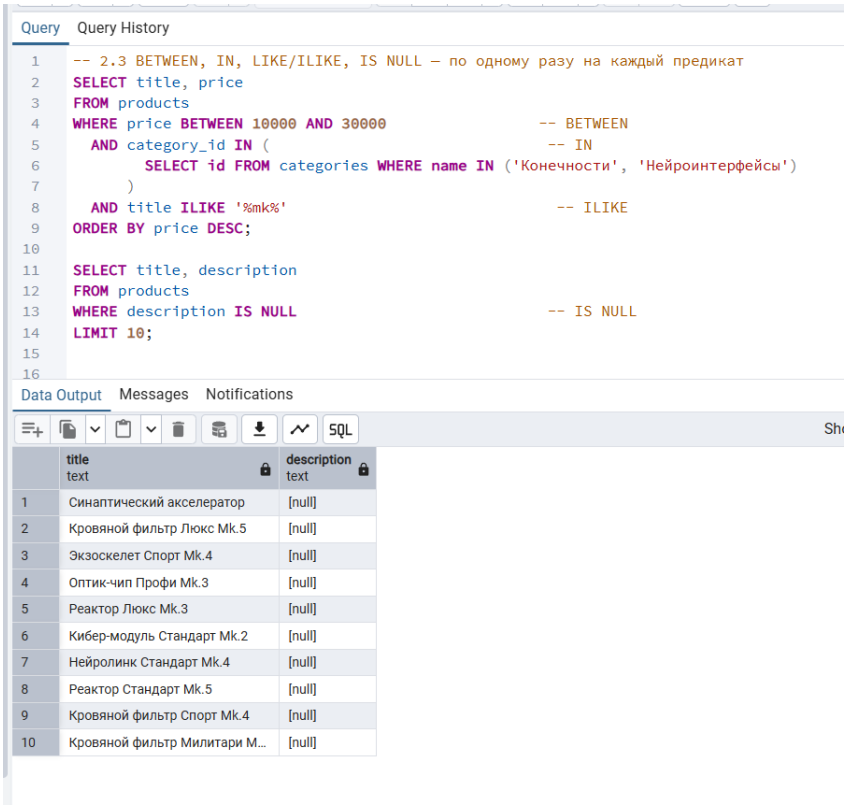
Total rows: 273 | Query complete 00:00:00.093

2.3 BETWEEN, IN, ILIKE и IS NULL

Первый запрос демонстрирует три предиката сразу: BETWEEN отбирает товары в ценовом диапазоне, IN — товары из двух конкретных категорий (подзапрос по именам категорий вместо жёстко прописанных id), ILIKE — товары, в названии которых встречается «mk» без учёта регистра. Второй запрос демонстрирует IS NULL: в процессе наполнения примерно 15% товаров намеренно оставлены без описания, чтобы предикат был содержательным, а не просто синтаксически верным.

```
SELECT title, price
FROM products
WHERE price BETWEEN 10000 AND 30000                  -- BETWEEN
  AND category_id IN (                                -- IN
        SELECT id FROM categories
        WHERE name IN ('Конечности', 'Нейроинтерфейсы')
      )
  AND title ILIKE '%mk%'                              -- ILIKE
ORDER BY price DESC;

SELECT title, description
FROM products
WHERE description IS NULL                             -- IS NULL
LIMIT 10;
```



Query | Query History

```
1  -- 2.3 BETWEEN, IN, LIKE/ILIKE, IS NULL — по одному разу на каждый предикат
2  SELECT title, price
3  FROM products
4  WHERE price BETWEEN 10000 AND 30000                -- BETWEEN
5    AND category_id IN (                              -- IN
6          SELECT id FROM categories WHERE name IN ('Конечности', 'Нейроинтерфейсы')
7        )
8    AND title ILIKE '%mk%'                            -- ILIKE
9  ORDER BY price DESC;
10
11 SELECT title, description
12 FROM products
13 WHERE description IS NULL                           -- IS NULL
14 LIMIT 10;
15
16
```

Data Output | Messages | Notifications

SQL

	title text	description text
1	Синаптический акселератор	[null]
2	Кровяной фильтр Люкс Mk.5	[null]
3	Экзоскелет Спорт Mk.4	[null]
4	Оптик-чип Профи Mk.3	[null]
5	Реактор Люкс Mk.3	[null]
6	Кибер-модуль Стандарт Mk.2	[null]
7	Нейролинк Стандарт Mk.4	[null]
8	Реактор Стандарт Mk.5	[null]
9	Кровяной фильтр Спорт Mk.4	[null]
10	Кровяной фильтр Милитари М...	[null]

На скриншоте виден текст обоих запросов и результат второго (IS NULL) — 10 товаров без описания. Для полноты картины желательно отдельно выполнить и заскриншотить только первый запрос (BETWEEN/IN/ILIKE), чтобы показать его собственный результат, а не только текст

2.4 Сортировка по двум столбцам в разных направлениях

Строки сначала группируются по возрастанию category_id, а внутри каждой категории — по убыванию цены. Поскольку товаров в каждой категории около 150, все 30 строк лимита приходятся на первую по возрастанию категорию (id = 5, «Оптические системы») — это не ошибка, а следствие LIMIT при большом количестве строк на одно значение первого ключа сортировки.

```
SELECT title, category_id, price
FROM products
ORDER BY category_id ASC, price DESC
LIMIT 30;
```



Query | Query History

```
2  -- 2.4 Сортировка по двум столбцам в разных направлениях
3  SELECT title, category_id, price
4  FROM products
5  ORDER BY category_id ASC, price DESC
6  LIMIT 30;
```

Data Output | Messages | Notifications

	title text	category_id bigint	price numeric (12,2)
1	Кровяной фильтр Стелс Mk.4	5	24676.50
2	Нейролинк Стандарт Mk.4	5	24491.23
3	Кибер-модуль Профи Mk.3	5	24392.09
4	Кибер-модуль Милитари Mk.3	5	24386.62
5	Оптик-чип Стандарт Mk.4	5	23738.93
6	Нейролинк Люкс Mk.3	5	23652.08
7	Нейролинк Спорт Mk.5	5	23537.47
8	Экзоскелет Спорт Mk.3	5	23487.12
9	Кибер-модуль Профи Mk.1	5	23436.47
10	Мышечный каркас Спорт Mk.3	5	23252.41
11	Мышечный каркас Спорт Mk.2	5	23180.64
12	Нейролинк Стандарт Mk.3	5	23004.90
13	Реактор Стандарт Mk.2	5	22961.63
14	Кибер-модуль Люкс Mk.1	5	22627.61
15	Мышечный каркас Милитари …	5	22232.11
16	Реактор Стелс Mk.2	5	21966.11
17	Кибер-модуль Милитари Mk.4	5	21831.68
18	Оптик-чип Стелс Mk.4	5	21683.78
19	Мышечный каркас Стандарт М…	5	21575.08
20	Реактор Профи Mk.1	5	21263.91
21	Нейролинк Спорт Mk.5	5	21183.14
22	Экзоскелет Милитари Mk.3	5	21058.91
23	Реактор Профи Mk.2	5	20907.59
24	Кибер-модуль Спорт Mk.5	5	20721.77
25	Кибер-модуль Стандарт Mk.3	5	20692.24
26	Нейролинк Люкс Mk.5	5	20470.38
27	Дермал-слой Милитари Mk.2	5	20314.34
28	Реактор Милитари Mk.5	5	19712.34
29	Кровяной фильтр Профи Mk.4	5	19708.01
30	Кибер-модуль Милитари Mk.3	5	19682.75

Total rows: 30 | Query complete 00:00:00.075

2.5 Постраничный вывод (LIMIT/OFFSET)

Запрос возвращает вторую «страницу» результатов по 20 записей — первые 20 самых дорогих активных товаров пропущены через OFFSET 20, а LIMIT 20 ограничивает вывод следующей порцией.

```
SELECT title, price
FROM products
WHERE is_active
ORDER BY price DESC, title ASC
LIMIT 20 OFFSET 20;
```

Query | Query History

```
1
2  -- 2.5 Постраничный вывод (LIMIT/OFFSET) — вторая страница по 20 записей
3  SELECT title, price
4  FROM products
5  WHERE is_active
6  ORDER BY price DESC, title ASC
7  LIMIT 20 OFFSET 20;
```

Data Output | Messages | Notifications

	title text	price numeric (12,2)
1	Кибер-модуль Профи Mk.2	24388.18
2	Кибер-модуль Милитари М...	24386.62
3	Кибер-модуль Спорт Mk.2	24325.16
4	Дермал-слой Профи Mk.3	24264.08
5	Экзоскелет Милитари Mk.1	24260.45
6	Кровяной фильтр Стелс Mk.2	24248.80
7	Нейролинк Профи Mk.5	24209.29
8	Дермал-слой Стандарт Mk.5	24203.45
9	Нейролинк Люкс Mk.3	24187.15
10	Дермал-слой Стандарт Mk.1	24162.44
11	Экзоскелет Люкс Mk.2	24156.86
12	Экзоскелет Спорт Mk.4	24137.51
13	Экзоскелет Спорт Mk.3	24074.88
14	Мышечный каркас Люкс М...	24044.22
15	Оптик-чип Стелс Mk.3	23980.67
16	Мышечный каркас Профи ...	23955.44
17	Кровяной фильтр Люкс Mk.2	23950.77
18	Кровяной фильтр Профи М...	23937.79
19	Экзоскелет Спорт Mk.5	23926.04
20	Нейролинк Стелс Mk.2	23911.17

Total rows: 20 | Query complete 00:00:00.106

3. ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №2:

— все 10 таблиц схемы KIIB наполнены согласованными тестовыми данными, товары и заказы — в объёме свыше 1000 строк через generate_series;

— написан запрос с DISTINCT и псевдонимами столбцов;

— написан запрос с фильтрацией по нескольким условиям через AND/OR со скобками;

— продемонстрированы предикаты BETWEEN, IN, ILIKE и IS NULL — каждый по одному разу, на содержательных данных;

— написан запрос с сортировкой по двум столбцам в разных направлениях;

— реализован постраничный вывод через LIMIT/OFFSET.

Наполнение таблиц:

```
-- TRUNCATE TABLE payments, order_items, reviews, orders, stock,
--     products, addresses, sellers, categories, users
--     RESTART IDENTITY CASCADE;



-- ============================================================
-- 1. ПОЛЬЗОВАТЕЛИ (users)
-- ============================================================

-- именные аккаунты (в т.ч. владельцы будущих продавцов)
INSERT INTO users (username, email) VALUES
    ('v_merc',           'v@afterlife.nc'),
    ('johnny_silverhand', 'johnny@samurai.nc'),
    ('jackie_w',         'jackie@heywood.nc'),
    ('judy_alvarez',     'judy@moxes.nc'),
    ('panam_palmer',     'panam@aldecaldos.nc'),
    ('viktor_vektor',    'vik@ripperdoc.nc'),
    ('rogue_amendiares', 'rogue@afterlife.nc'),
    ('river_ward',       'river@ncpd.nc'),
    ('corp_arasaka_hq',  'admin@arasaka.corp'),
    ('corp_militech_hq', 'admin@militech.corp'),
    ('corp_kiroshi_hq',  'admin@kiroshi.corp'),
    ('corp_biotech_hq',  'admin@biotechnica.corp'),
    ('corp_zetatech_hq', 'admin@zetatech.corp'),
    ('corp_dynalar_hq',  'admin@dynalar.corp'),
    ('corp_kangtao_hq',  'admin@kangtao.corp'),
```

```
    ('corp_zheng_hq',     'admin@shengzheng.corp');


-- массовое наполнение — обычные жители Найт-Сити
INSERT INTO users (username, email)
SELECT
    'nc_citizen_' || g,
    'citizen' || g || '@nightcity.nc'
FROM generate_series(1, 184) AS g;
-- итого 200 пользователей (16 именных + 184 сгенерированных)




-- ============================================================
-- 2. ПРОДАВЦЫ (sellers) — корпорации, клиники, частные мастера
-- ============================================================


INSERT INTO sellers (user_id, name, license_number, rating)
SELECT id, 'Arasaka Cyberware Division',          'LIC-AR-001', 4.8 FROM users WHERE username = 'corp_arasaka_hq'
UNION ALL
SELECT id, 'Militech Tactical Augments',          'LIC-MT-002', 4.6 FROM users WHERE username = 'corp_militech_hq'
UNION ALL
SELECT id, 'Kiroshi Optics',                      'LIC-KI-003', 4.9 FROM users WHERE username = 'corp_kiroshi_hq'
UNION ALL
SELECT id, 'Biotechnica Organic Solutions',       'LIC-BT-004', 4.2 FROM users WHERE username = 'corp_biotech_hq'
UNION ALL
```

```
SELECT id, 'Zetatech Neural Systems',          'LIC-ZT-005', 4.4 FROM users WHERE username = 'corp_zetatech_hq'
UNION ALL
SELECT id, 'Dynalar Structural Reinforcement',  'LIC-DY-006', 4.0 FROM users WHERE username = 'corp_dynalar_hq'
UNION ALL
SELECT id, 'Kang Tao Combat Systems',          'LIC-KT-007', 4.3 FROM users WHERE username = 'corp_kangtao_hq'
UNION ALL
SELECT id, 'Viktor"s Clinic (частный рипер-док)', NULL,       4.95 FROM users WHERE username = 'viktor_vektor';



-- ============================================================
-- 3. КАТЕГОРИИ (categories) — иерархия
-- ============================================================

-- корневые категории
INSERT INTO categories (name, description) VALUES
    ('Кибернетические импланты', 'Механические и электронные аугментации тела'),
    ('Био-модификации',          'Органические и генетически изменённые модули'),
    ('Программное обеспечение',  'Дейталки, демоны, квикхаки'),
    ('Экипировка рипер-дока',    'Инструменты и расходники для установки имплантов');

-- дочерние категории кибернетических имплантов
INSERT INTO categories (parent_id, name, description)
SELECT id, 'Оптические системы', 'Импланты глаз и зрительные модули'
FROM categories WHERE name = 'Кибернетические импланты'
```

```
UNION ALL

SELECT id, 'Нейроинтерфейсы', 'Мозговые импланты, разъёмы и акселераторы'

FROM categories WHERE name = 'Кибернетические импланты'

UNION ALL

SELECT id, 'Конечности', 'Кибер-руки, кибер-ноги, встроенное оружие'

FROM categories WHERE name = 'Кибернетические импланты'

UNION ALL

SELECT id, 'Кибердеки', 'Устройства для хакинга и квикхаков'

FROM categories WHERE name = 'Кибернетические импланты';


-- дочерние категории био-модификаций

INSERT INTO categories (parent_id, name, description)

SELECT id, 'Дермальные покрытия', 'Подкожная броня и защитные слои кожи'

FROM categories WHERE name = 'Био-модификации'

UNION ALL

SELECT id, 'Мышечные каркасы', 'Усиление физической силы и выносливости'

FROM categories WHERE name = 'Био-модификации'

UNION ALL

SELECT id, 'Кровеносная система', 'Модули очистки крови и ускоренной регенерации'

FROM categories WHERE name = 'Био-модификации';




-- ============================================================

-- 4. ТОВАРЫ (products)

-- ============================================================


-- 4.1. флагманские товары (узнаваемые позиции — для наглядности примеров)
```

```
INSERT INTO products (seller_id, category_id, title, description, price, is_active)
SELECT s.id, c.id, 'Kiroshi Optics Mk.3', 'Улучшенный зрительный имплант с зумом и записью', 12500.00, TRUE
FROM sellers s, categories c
WHERE s.name = 'Kiroshi Optics' AND c.name = 'Оптические системы'
UNION ALL
SELECT s.id, c.id, 'Sandevistan Mk.4', 'Импланты замедления времени для рефлекторных реакций', 45000.00, TRUE
FROM sellers s, categories c
WHERE s.name = 'Militech Tactical Augments' AND c.name = 'Нейроинтерфейсы'
UNION ALL
SELECT s.id, c.id, 'Mantis Blades', 'Выдвижные клинки, встроенные в предплечья', 38000.00, TRUE
FROM sellers s, categories c
WHERE s.name = 'Arasaka Cyberware Division' AND c.name = 'Конечности'
UNION ALL
SELECT s.id, c.id, 'Gorilla Arms', 'Усиленные кибер-руки для рукопашного боя', 27500.00, TRUE
FROM sellers s, categories c
WHERE s.name = 'Militech Tactical Augments' AND c.name = 'Конечности'
UNION ALL
SELECT s.id, c.id, 'Cyberdeck Tetratronic Rippersaw', 'Кибердека для продвинутых квикхаков', 32000.00, TRUE
FROM sellers s, categories c
WHERE s.name = 'Zetatech Neural Systems' AND c.name = 'Кибердеки'
UNION ALL
SELECT s.id, c.id, 'Синаптический акселератор', 'Ускоряет реакцию нервной системы', 15800.00, TRUE
FROM sellers s, categories c
WHERE s.name = 'Zetatech Neural Systems' AND c.name = 'Нейроинтерфейсы'
```

```
UNION ALL

SELECT s.id, c.id, 'Subdermal Armor', 'Подкожная бронепластина', 9800.00, TRUE

FROM sellers s, categories c

WHERE s.name = 'Biotechnica Organic Solutions' AND c.name = 'Дермальные покрытия'

UNION ALL

SELECT s.id, c.id, 'Biomonitor', 'Мониторинг жизненных показателей и авто-инъекции', 4200.00, TRUE

FROM sellers s, categories c

WHERE s.name = 'Biotechnica Organic Solutions' AND c.name = 'Кровеносная система'

UNION ALL

SELECT s.id, c.id, 'Bionic Legs "Fenrir"', 'Усиленные ноги для прыжков и бега', 21000.00, TRUE

FROM sellers s, categories c

WHERE s.name = 'Dynalar Structural Reinforcement' AND c.name = 'Конечности'

UNION ALL

SELECT s.id, c.id, 'Reinforced Tendons', 'Армированные сухожилия для увеличения силы хвата', 6400.00, FALSE

FROM sellers s, categories c

WHERE s.name = 'Dynalar Structural Reinforcement' AND c.name = 'Мышечные каркасы';


-- 4.2. массовое наполнение до объёма, достаточного для аналитики

INSERT INTO products (seller_id, category_id, title, description, price, is_active)

SELECT

   sellers_arr[1 + floor(random() * array_length(sellers_arr, 1))::int],

   cats_arr[1 + floor(random() * array_length(cats_arr, 1))::int],

   prefix[1 + floor(random() * array_length(prefix, 1))::int]

      || ' ' || suffix[1 + floor(random() * array_length(suffix, 1))::int]

      || ' Mk.' || (1 + floor(random() * 5))::int,

   'Серийная модель для массового рынка',
```

```
    round((random() * 24000 + 800)::numeric, 2),
    (random() > 0.08)
FROM generate_series(1, 990) AS g,
     (SELECT array_agg(id) AS sellers_arr FROM sellers) s,
     (SELECT array_agg(id) AS cats_arr FROM categories WHERE parent_id IS NOT NULL) c,
     (SELECT ARRAY['Нейролинк', 'Кибер-модуль', 'Дермал-слой', 'Оптик-чип',
                   'Мышечный каркас', 'Кровяной фильтр', 'Реактор', 'Экзоскелет']
        AS prefix) pr,
     (SELECT ARRAY['Стандарт', 'Профи', 'Милитари', 'Стелс', 'Спорт', 'Люкс']
        AS suffix) su;
-- итого около 1000 товаров (10 флагманских + 990 серийных)



-- ============================================================
-- 5. СКЛАДСКИЕ ОСТАТКИ (stock)
-- ============================================================

INSERT INTO stock (product_id, quantity, warehouse_location)
SELECT
    id,
    floor(random() * 250)::int,
    (ARRAY['Watson, склад №1', 'Westbrook, склад №2', 'Heywood, склад №3',
           'Pacifica, склад №4', 'City Center, склад №5'])[1 + floor(random() * 5)::int]
FROM products;



-- ============================================================
```

```
-- 6. ЗАКАЗЫ (orders)
-- ============================================================


INSERT INTO orders (user_id, status, created_at)
SELECT
    users_arr[1 + floor(random() * array_length(users_arr, 1))::int],
    (ARRAY['pending', 'confirmed', 'completed', 'cancelled'])[1 + floor(random() * 4)::int],
    now() - (random() * interval '365 days')
FROM generate_series(1, 1200) AS g,
     (SELECT array_agg(id) AS users_arr FROM users) u;



-- ============================================================
-- 7. ПОЗИЦИИ ЗАКАЗА (order_items)
-- ============================================================


-- на каждый заказ — до 3 попыток добавить случайную позицию;
-- часть отсеется по UNIQUE/PK (order_id, product_id) — заказ
-- получит от 1 до 3 разных товаров, что и требуется
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT
    picks.order_id,
    picks.product_id,
    picks.quantity,
    p.price
FROM (
    SELECT
```

```
        o.id AS order_id,
        arr.ids[(1 + floor(random() * array_length(arr.ids, 1)))::int] AS product_id,
        (1 + floor(random() * 3))::int AS quantity
    FROM orders o
    CROSS JOIN generate_series(1, 3) AS item_no
    CROSS JOIN (SELECT array_agg(id) AS ids FROM products) arr
) picks
JOIN products p ON p.id = picks.product_id
ON CONFLICT (order_id, product_id) DO NOTHING;



-- ============================================================
-- 8. ОТЗЫВЫ (reviews)
-- ============================================================

-- 900 попыток случайных пар (пользователь, товар); часть отсеется
-- из-за ограничения UNIQUE (user_id, product_id) — это ожидаемо
INSERT INTO reviews (user_id, product_id, score, comment, created_at)
SELECT
    users_arr[1 + floor(random() * array_length(users_arr, 1))::int],
    products_arr[1 + floor(random() * array_length(products_arr, 1))::int],
    1 + floor(random() * 5)::int,
    (ARRAY[
        'Стабильно работает, отторжения не было.',
        'Пришлось донастраивать у рипер-дока, но в целом доволен.',
        'Совместимость с моей ОС так себе, будьте аккуратны.',
        'Лучшее вложение эдди за последний год.',
```

```
        NULL,
        'Гарантийный ремонт прошёл без проблем.'
    ])[1 + floor(random() * 6)::int],
    now() - (random() * interval '300 days')
FROM generate_series(1, 900) AS g,
     (SELECT array_agg(id) AS users_arr FROM users) u,
     (SELECT array_agg(id) AS products_arr FROM products) pr
ON CONFLICT (user_id, product_id) DO NOTHING;



-- ============================================================
-- 9. ПЛАТЕЖИ (payments)
-- ============================================================

-- платёж создаётся только для заказов, у которых есть позиции
-- и статус отличен от 'pending' (по заказу ещё не заплатили)
INSERT INTO payments (order_id, amount, payment_method, payment_date, status)
SELECT
    o.id,
    items.total,
    (ARRAY['credit_chip', 'crypto', 'corp_account'])[1 + floor(random() * 3)::int],
    o.created_at + (random() * interval '2 days'),
    CASE o.status
        WHEN 'completed' THEN 'completed'
        WHEN 'cancelled' THEN 'refunded'
        ELSE 'completed'
    END
```

```
FROM orders o
JOIN LATERAL (
    SELECT SUM(quantity * unit_price) AS total
    FROM order_items
    WHERE order_id = o.id
) items ON items.total > 0
WHERE o.status <> 'pending';



-- ============================================================
-- 10. АДРЕСА (addresses)
-- ============================================================

-- адреса пользователей (доставка)
INSERT INTO addresses (user_id, address_line, city, is_default)
SELECT
    id,
    'Мегабашня ' || (1 + floor(random() * 40))::int || ', уровень ' || (1 + floor(random() * 90))::int,
    (ARRAY['Watson', 'Westbrook', 'Heywood', 'Pacifica', 'Santo Domingo', 'City Center'])[1 + floor(random() * 6)::int],
    TRUE
FROM users
WHERE random() < 0.6;

-- адреса клиник продавцов
INSERT INTO addresses (seller_id, address_line, city, is_default)
SELECT
```

```
    id,
    'Клиника, корп-квартал ' || (1 + floor(random() * 12))::int,
    (ARRAY['Watson', 'Westbrook', 'City Center'])[1 + floor(random() * 3)::int],
    TRUE
FROM sellers;
```